Einführung in die Programmierung I
Einleitung

PD Dr. Andreas Paffenholz
Erik Jansen

TECHNISCHE UNIVERSITÄT DARMSTADT

WiSe 26/27
19.10.2026

Die folgende Einleitung behandelt die Arbeit mit der Kommandozeile, gegebenfalls die Arbeit auf einem Windows-Subsystem für Linux, sowie das Kompilieren eines C-Programmms mittels `gcc`.

Einleitung

Nutzeraccount, Linux und aller Anfang ist hürdenreich

In der ersten Vorlesung erhalten Sie einen Nutzeraccount und das dazugehörige Passwort um in den Poolräumen des Fachbereich Mathematik arbeiten zu können.

Notieren oder fotografieren Sie sich Nutzernamen und Passwort!

Auf den PC's in den Poolräumen des Fachbereichs wird GNU/Linux als Betriebssystem verwendet. Für die Ziele der Vorlesung ist Arbeiten mit der Textkonsole immer wieder notwendig. Machen Sie sich daher zunächst mit der Konsole/dem Terminal vertraut[1].

Das Tool `man` ist ein eingebautes Handbuch, welches zu vielen Befehlen und Funktionen ausführliche Informationen zur Verfügung stellt. Im Verlauf des Semesters könnte „`man`“ Ihnen noch eine großartige Hilfe werden.

Editoren

Bis hierher haben wir lediglich mit der Konsole auf dem System umhergeschaut. Zum Programmieren müssen Sie Textdateien mit Quellcode anlegen und mit geeigneten Werkzeugen daraus ausführbare Dateien erzeugen lassen.

Als Editor für Textdateien empfehlen wir Ihnen einen der folgenden:

- `Visual Studio Code` (Unsere Empfehlung) (Befehl: `code`)
- `gedit`
- `nano`
- `vim` bzw. `gvim`
- `emacs`

[1] Sie sind eingeladen sich selbstständig auch über das in der Vorlesung Genannte hinaus mit GNU/Linux und den diversen Desktop-Umgebungen bzw. der Bash auseinander zu setzen! Uns fehlt hier lediglich die Zeit dazu. Wir haben Vertrauen in Ihren Umgang mit (Computer-)Technik :).

Diese können Sie einfach auf der Konsole aufrufen. Manche starten im Konsolenfenster, manche starten eine eigenständige GUI[2] wie z.B. Visual Studio Code.

Unter Angabe eines Dateinamens nach dem Programmnamen eines Editors öffnet der Editor direkt die angegebene Datei, bzw. legt diese an.

Probieren Sie die Editoren aus und probieren Sie, eine Datei mit beliebigem Inhalt anzulegen und abzuspeichern. Beenden Sie den Editor anschließend wieder.

Einige Befehle im Terminal werden nicht sofort beendet, sodass das Terminal in deren Ausführung verharrt und keine weiteren Befehle eingegeben werden können. Dies kann z.B. die Ausführung eines Programmes sein. In der Regel um Zugang zum Terminal zu haben, während ein Programm ausgeführt wird, müssen Sie nach dem Befehl das Und-Zeichen (&) einfügen, z.B. `firefox&`, `emacs&`, etc. Wenn Sie dies vergessen, müssen Sie Ihr laufendes Programm erst beenden, indem Sie `strg+c` eingeben.

Commandline-Tools

Bei der Arbeit mit Linux sind viele Werkzeuge sogenannte Commandline-Tools, das heißt ihr Aufruf erfolgt über die sogenannte Konsole (auch “Terminal”, “bash” oder “shell”). Arbeiten Sie mit einem Windows-Subsystem für Linux, so landen Sie nach Aufruf desselben automatisch in einem Terminal. Die wichtigsten Konsolenbefehle haben wir auf der letzten Seite dieses Dokumentes für Sie zusammengestellt.

Öffnen Sie die Konsole und geben Sie die folgenden Befehle ein, um sich mit der Konsolen bzw. Terminalumgebung vertraut zu machen.

1. Überprüfen Sie, ob Sie im Homeverzeichnis sind indem Sie `pwd` (**p**rint **w**ork **d**irectory) eingeben. Wenn Sie im Homeverzeichnis sind, sollte in der Konsole `computer_name:~$` (Linux) oder `computer_name:~` (MacOS) als Antwort zurückgegeben werden. Diese sogenannte Eingabeaufforderung kann allerdings auch abhängig von ihren Einstellungen variieren (z.B. wird oft das aktuelle Verzeichnis oder die Uhrzeit angezeigt). Wenn Sie nicht im Homeverzeichnis sind, gelangen Sie durch Eingabe von `cd ~` oder einfach `cd` ins Homeverzeichnis.
2. Geben Sie `ls` ein, um den Inhalt des Homeverzeichnisses anzeigen zu lassen.
3. Erstellen Sie ein neues Verzeichnis mit dem Namen `EinfProg`, indem Sie `mkdir EinfProg` eingeben. Geben Sie erneut `ls` ein, um sicherzustellen, dass der Ordner erstellt wurde.
4. Erstellen Sie zwei Unterverzeichnisse von `EinfProg`, die Sie `Uebung_1` und `Uebung_2` nennen, indem Sie `mkdir EinfProg/Uebung_1` und `mkdir EinfProg/Uebung_2` eingeben. Gehen Sie in das Verzeichnis `EinfProg`, indem Sie `cd EinfProg` eingeben und überprüfen Sie mit `ls`, dass die Unterverzeichnisse erstellt wurden.
5. Gehen Sie mittels `cd` zurück in das Homeverzeichnis und erstellen Sie zwei Unterverzeichnisse von `Uebung_1`, welche Sie `test_1` und `test_2` nennen, indem Sie `mkdir` in Form von `mkdir EinfProg/Uebung_1/test_1` benutzen.
6. Um zu überprüfen, dass die zwei Unterverzeichnisse erstellt wurden, geben Sie `ls EinfProg/Uebung_1` ein. Alternativ können Sie auch `ls -R` eingeben, was alle Verzeichnisse rekursiv auflistet.
7. Sie können es vermeiden immer die gleichen Befehle einzugeben, indem Sie die Pfeiltasten `hoch` und `runter` verwenden, um die vorher eingegebenen Befehle zu sehen. Ein weiterer nützlicher Tipp ist es, die `TAB` Taste für die Autovervollständigung von Datei- oder Verzeichnisnamen zu benutzen.
8. Gehen Sie in das `EinfProg` Verzeichnis und überprüfen Sie mit dem `pwd` Befehl, ob Sie im richtigen Verzeichnis sind. Gehen Sie als nächstes in das `test_1` Verzeichnis und überprüfen Sie erneut, dass Sie sich im richtigen Verzeichnis befinden.

[2] GUI = Graphical User Interface. Grammatikalisch müsste man vielleicht „ein eigenständiges GUI“ sagen, wir sehen das nicht so eng.

9. Sie können in der Verzeichnishierarchie in das übergeordnete Verzeichnis gehen, indem Sie den Befehl `cd ..` eingeben (der Abstand dazwischen ist wichtig). In unserem Beispiel sollte der Befehl `cd ..` Sie in das Verzeichnis `Uebung_1` bringen. Ein einzelner Punkt (`.`) steht für das aktuelle Verzeichnis und zwei Punkte (`..`) für das übergeordnete Verzeichnis. Der Befehl `cd .` bewirkt also nichts, außer dass wir im aktuellen Verzeichnis bleiben.
10. Sie können in einem Schritt in das `test_2` Verzeichnis gehen, unabhängig davon wo Sie aktuell sind durch den Befehl `cd ~/EinfProg/Uebung_1/test_2`.
    Wenn Sie im `test_2` Verzeichnis sind, können Sie mit dem Befehl `cd ../test_1` in das Verzeichnis `test_1` wechseln.

Falls Sie Linux oder MacOS benutzen springen Sie nun bitte gleich zu Schritt 17.

Für Nutzer des Linux-Subsystems von Windows[3]:

Der Homeordner Ihres Linux-Subsystems ist von Windows aus unter einem unangenehmen Pfad, etwa

```
C:\Users\username\AppData\Local\Packages\CanonicalGroupLimited.Ubuntu18.04onWindows_79
                    rhkp1fndgsc\LocalState\rootfs\home\username
```

erreichen. Das ist unpraktisch.

11. Wechseln Sie darum mittels `cd /mnt/c/Users/username` zu Ihrem Windows-Nutzer-Ordner, wobei Sie wieder `c` ggf. durch den Namen der Partition ersetzten auf welcher Windows installiert ist und `username` durch ihren Windowsnutzernamen.
12. Erstellen Sie mit `mkdir EinfProg` den Ordner `EinfProg`

Diesen Ordner erreichen Sie von Windows aus unter `C:\Users\username\EinfProg`

13. Updaten Sie zunächst das Advanced Packaging Tool (APT) mittels `sudo apt update && sudo apt upgrade`
14. Installieren Sie die Gnu Compiler Collection mittels Eingabe des Befehls
    `sudo apt-get install build-essential gdb`
    Prüfen Sie die Installation durch Eingabe von `gcc --version`
15. Genauso installieren Sie die Gnu Compiler Collection mittels Eingabe des Befehls `sudo apt install git-all` und prüfen die Installation durch Eingabe von `git --version`
15. Aktivieren Sie außerdem die copy&paste-Funktion zwischen Linux und Windows, indem Sie auf die Kopfzeile des Terminals rechtsklicken, ”Eigenschaften“ auswählen und den entsprechenden Haken setzen.
16. Wechseln Sie nun in den Ordner `EinfProg`
17. Kopieren Sie mittels

    `git clone https://gitlab.hefdi.de/tuda/einf_prog_i/2026/material/tutorials/demo.git`

    unser Beispielprojekt in das automatisch angelegte Verzeichnis `demo`. Hierfür benötigen Sie eine Internetverbindung, da git dieses Verzeichnis das Abbild eines online(remote) existierenden Verzeichnisses auf der Platform Gitlab darstellen soll und hierfür eine Kommunikation mit Gitlab möglich sein muss. Für mehr Informationen zu Git schauen Sie im Moodle-Kurs unter "Weitere Materialien" in die "Technische Hilfen".
18. Wechseln Sie in das Verzeichnis `demo` und prüfen Sie dass die C-Quellcode Datei `hello_world.c` vorhanden ist.
19. Kompilieren Sie den Quellcode mit dem Befehl `gcc hello_world.c -o hello` . Sofern Ihr Programm korrekt kompiliert wurde, wurde die ausführbare Datei `hello` erstellt. Sie können diese mit den Befehl `./hello` ausführen.
20. Sie können während des Kompilierens weitere Optionen hinzufügen, z.B. `gcc hello_world.c -Wall -pedantic`, um zusätzliche Warnmeldungen vom Compiler zu erhalten. Benutzen Sie den `man gcc` Befehl um mehr über diese Optionen zu erfahren.

[3] Für die korrekte Installation von Compiler und Git auf Ihren eigenen Windows oder Mac-Rechner sind Sie selbst verantwortlich. Sie finden jedoch unter Vorlesungsmaterialien/Technische Hilfen in Moodle eine Anleitung

21. Öffnen Sie die Datei `hello_world.c` mit Ihrem installiertem Texteditor (Nutzen Sie ein Windows-Subsystem, so tun tun Sie das unter Windows). Ändern Sie die Zeile

    ```
    printf("Hello, World!\n");
    ```

    in

    ```
    printf("Ich kann nun C-Dateien kompilieren.\n");
    ```

    und wiederholen Sie das Kompilieren und Ausführen ab Schritt 19. Das `\n` ist ein Zeilentrenner, sodass eine neue Zeile in der Ausgabe danach beginnt. Die Zeichen `\n` selbst werden nicht ausgegeben. Probieren Sie das Kompilieren und Ausführen auch gerne einmal ohne `n` und beobachten Sie den Unterschied.

Cheat Sheet: Terminal Befehle

Die wichtigsten Befehle für die Konsole haben wir hier zusammengestellt:

- `whoami` → (**WHO AM I**) Zeigt den Benutzernamen des aktuellen Benutzers an
- `pwd` → (**P**rint **W**orking **D**irectory) Zeigt den Namen des aktuellen Verzeichnisses an
- `ls` → (**Li**i**S**ting) Zeigt den Inhalt des aktuellen Verzeichnisses an
- `cd` → (**C**hange **D**irectory) Wechselt das aktuelle Verzeichnis
- `mkdir` → (**M**a**K**e **DIR**ectory) Erzeugt ein neues Verzeichnis
- `cp` → (**C**o**P**y) Kopiert Dateien und Verzeichnisse
- `mv` → (**M**o**V**e) Verschiebt und benennt Dateien und Verzeichnisse um
- `rm` → (**R**e**M**ove) Löscht Dateien
- `rmdir` → (**R**e**M**ove **DIR**ectory) Löscht Verzeichnisse und enthaltene Dateien
- `gcc` → (**G**nu **C** **C**ompiler) Kompiliert .c Dateien
- `man` → (**MAN**ual) Hilfe für Unix Befehle und C Funktionen (Hinweis: `q` beendet das Anzeigeprogramm)

Sehr häufig geben Programme mit der Option `--help` oder `-h` eine Hilfe aus. Wenn Sie Hilfe zu einem Befehl und seinen Optionen benötigen, können Sie das sehr hilfreiche Tool `man` verwenden, z.B. `man ls` oder `man gcc`.